

Benchmarking Requirement-to-Architecture Generation with Hybrid Evaluation

Minxiao Li, Li Zhang, Shuying Yan, Fang Liu, Liehao Li, Yang Liu, and Xiaoli Lian

State Key Laboratory of Complex & Critical Software Environment

School of Computer Science and Engineering

Beihang University, Beijing, China

liminxiao@buaa.edu.cn, fangliu@buaa.edu.cn

Abstract—Software architecture serves as the blueprint of a software system, capturing high-level structural decisions that shape downstream implementation and system quality. Despite this central role, generating architecture designs from requirement documents remains underexplored, with limited task-specific benchmarks for generation or rigorous evaluation. To bridge this gap, we introduce R2ABENCH, a benchmark for requirements-to-architecture (R2A) reasoning. R2ABENCH contains 68 projects with human-validated references collected from both educational-style settings and GitHub repositories. Each project is packaged as a unified instance that provides a structured software requirements specification (SRS), a reference architecture view in PlantUML and the corresponding source requirement artifacts. To evaluate generated architecture views, we design a layered hybrid evaluation framework spanning syntax validation, structural graph diagnostics, and semantic and evidence-based architecture scoring. Using this framework, we conduct a comprehensive empirical study of state-of-the-art LLMs and agents on R2ABENCH. We find that current systems can often generate syntactically valid and readable architecture views, but still struggle with relation-level architecture modeling across both subsets: component identification is substantially stronger than edge recovery, and edge hallucination is the dominant structural failure mode. Semantically, structural fidelity does not guarantee requirement coverage or traceability, which emerge as the primary quality gaps. The associated data files are available at <https://figshare.com/s/01f0a5fb6243a6a60f23>.

Index Terms—Large Language Models, Software Architecture, Requirements Engineering

I. INTRODUCTION

Software architecture is a critical bridge between requirements and implementation [1], [2]. It describes the core components of a system, their responsibilities, hierarchical relationships, deployment logic, and interactions with external systems. Architecture views are essential artifacts for expressing and communicating such architectural decisions [3]. As defined by Rozanski and Woods [4], a view is a representation of one or more structural aspects of an architecture that explains how the architecture addresses stakeholder concerns. Through architecture views, complex systems can be decomposed into understandable representations for different concerns, supporting communication and collaboration among developers, architects, operators, and business stakeholders [5], [6].

Despite their importance, deriving a faithful architecture view from requirements remains a costly and inconsistent activity. It is far from a mechanical translation: an analyst must

reconcile functional requirements with non-functional requirements, technical constraints, domain rules, and architecturally significant requirements (ASRs), and synthesize them into a coherent structure of components, boundaries, dependencies, data stores, and external interfaces. Because this synthesis is performed manually, the requirements-to-architecture mapping is often produced inconsistently and with weak traceability between design decisions and the requirements that justify them. The resulting views are also hard to keep faithful: an analysis of 15,000 architecture views across 12,200 projects finds that roughly 75% are never updated after creation [7]. This makes requirements-to-architecture reasoning a natural target for automated support.

Large language models (LLMs) offer a new opportunity to address this long-standing problem. In recent years, generative AI has been increasingly adopted in code generation, testing, program repair, and software maintenance, and recent studies have begun to explore the use of LLMs to support architectural activities, ranging from design-decision support and architecture analysis to runtime architectural adaptation [8]. However, as noted by Esposito et al. [8], the capability of LLMs to generate architecture views remains largely underexplored. This task is not a straightforward extension of code generation. LLMs are strong at producing code snippets and completing local functions, but these are largely local, self-contained tasks. Architecture views instead demand system-level reasoning—understanding broad stakeholder concerns, selecting appropriate abstraction levels, recognizing architectural patterns, and reflecting quality-attribute requirements—so whether the coding strengths of LLMs carry over to this setting is far from obvious. Similarly, general-purpose coding agents with task decomposition, tool use, and memory have been applied to complex software engineering tasks, but it remains unclear whether they can reliably reason from requirements to architecture views.

Answering this question empirically calls for a benchmark: requirements paired with reference architecture views, together with a way to score a generated view against a reference. Existing resources do not meet this need. Prior work mostly targets text-to-model generation for individual artifact types from curated descriptions [9], [10], [11], while architecture-recovery resources extract views from existing source code rather than generating them from requirements [7], [12]. Studies that do generate architecture from requirements are typically

evaluated on a few case studies, without a reusable benchmark or reference views to compare against [13], [14]. Evaluation is a further open problem: architecture generation admits no single executable oracle, unlike repository-level coding benchmarks where executable validation provides a success signal [15], so text-based metrics and single-judge LLM checks are both inadequate [16], [17]. We compare existing benchmarks in detail in Section VII-C. The absence of both a dedicated benchmark and a reliable evaluation methodology jointly hinders progress in this area.

To address both gaps, we introduce R2ABENCH, a benchmark for requirements-to-architecture (R2A) reasoning. Specifically, R2ABENCH contains 68 projects with human-validated references drawn from educational settings and GitHub repositories. Each instance is organized around a shared SRS schema that normalizes requirements into functional requirements, non-functional requirements, technical constraints, domain rules, actors, data objects, external systems, and ASRs. Beyond this specification, each project provides a reference architecture view in PlantUML that can be parsed into a typed component graph, the corresponding source requirement artifacts, and requirement-to-evidence trace links. Building on this benchmark, and to overcome the evaluation difficulties above, we design a layered hybrid evaluation framework that combines syntax validation, structural graph diagnostics, semantic and evidence-driven architecture scoring, and human calibration, so that architecture quality is assessed through complementary signals rather than any single, biased metric.

Using R2ABENCH, our empirical study of state-of-the-art LLMs and agents shows that current systems generate syntactically valid and readable views but remain limited at relation-level modeling, identifying components far more reliably than they recover dependencies.

Our contributions are as follows:

- We introduce R2ABENCH, a benchmark for requirements-to-architecture reasoning, containing 68 projects with human-validated references from educational courses and GitHub repositories.
- We design a layered hybrid evaluation framework that combines syntax validation, structural graph diagnostics, semantic and evidence-driven architecture scoring.
- We conduct an extensive empirical study of state-of-the-art LLMs and agents on R2ABENCH, evaluating architecture generation across structural fidelity, semantic adequacy, and evidence traceability.

II. TASK FORMULATION

We frame R2A as a forward synthesis task: given a project p , the generator G derives an architecture view $\hat{\mathcal{V}}_p$ from a structured software requirements specification $\mathcal{S}_p$ alone, rather than recovering one from existing code:

$$\hat{\mathcal{V}}_p = G(\mathcal{S}_p). \quad (1)$$

For evaluation, each project is additionally annotated with a reference architecture view $\mathcal{V}_p^*$.

The input $\mathcal{S}_p$ is a normalized software requirements specification. To provide a standardized requirements representation, we structure the SRS with reference to ISO/IEC/IEEE 29148 [18]:

$$\mathcal{S}_p = \langle FR, NFR, TC, DR, A, D, ES, ASR \rangle, \quad (2)$$

where FR denotes functional requirements, NFR non-functional requirements, TC technical constraints, DR domain rules, A actors, D data objects, ES external systems, and ASR architecturally significant requirements. This representation provides a common task interface across projects, while retaining the functional, quality-attribute, and constraint information relevant to architectural decisions.

The output $\hat{\mathcal{V}}_p$ is an architecture view, represented as a typed component graph that captures the system's structural decomposition. Each view specifies the major architectural elements and their responsibilities, the relations among them, layers or boundaries, external systems, and key data stores. Concretely, we serialize $\hat{\mathcal{V}}_p$ as PlantUML code. We adopt PlantUML rather than image-based diagrams because its textual form can be generated directly by LLMs and, unlike free-form images, can be deterministically rendered for inspection and parsed into a graph for automatic evaluation.

III. DATA CONSTRUCTION

A. Overview

Building a benchmark for R2A reasoning faces a fundamental tension: evaluation requires clean, well-structured references with unambiguous ground truth, whereas software developed in practice rarely provides them. To address this, R2ABENCH pairs two complementary subsets. The educational subset (E-R2A) consists of graduate-level course projects whose requirements documents follow the ISO/IEC/IEEE 29148 standard and whose reference views were peer-reviewed in course settings; these properties make E-R2A well-suited for rigorous and reproducible evaluation. The subset of projects collected from GitHub (G-R2A), in contrast, consists of openly developed software whose architectures are more tightly coupled and whose requirements are not documented in any single, explicit specification. Together, the two subsets let R2ABENCH assess R2A capability under both idealized and repository-grounded conditions.

B. E-R2A: Educational-Project Benchmark Subset

E-R2A is the educational-project subset in R2ABENCH, targeting R2A scenarios where project architectures are relatively well-structured and requirements documents are comparatively clear. The data are collected from graduate-level software engineering course projects. Each project contains a software requirements analysis document, an architecture view, and the corresponding source requirement artifacts. The requirements documents were authored following the ISO/IEC/IEEE 29148 requirements-engineering standard [18], and both the documents and the diagrams were reviewed and graded as part of the course process, providing vetted reference artifacts for evaluation.

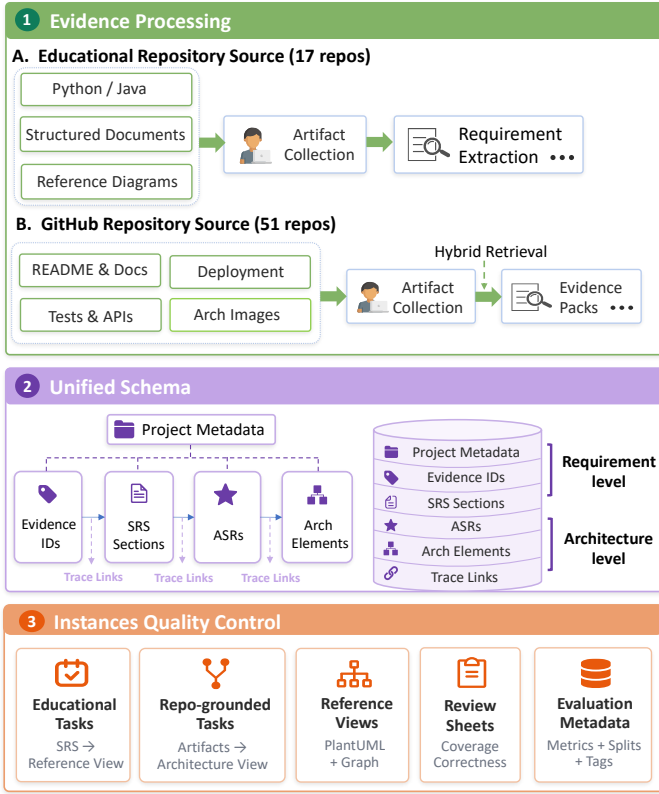


Fig. 1: Overview of the R2ABENCH construction pipeline: evidence processing, unified schema, and quality control.

To ensure the selected projects are suitable for architecture view generation and evaluation, we apply three criteria: ❶ the target programming language must account for more than 50% of the total code by bytes; ❷ the system must contain a complete and clear architecture view to serve as the reference artifact; and ❸ the system must maintain a clear front-end/back-end separation and a reasonably modular structure. Following these criteria, E-R2A contains 17 project samples.

The construction of E-R2A consists of three steps: requirements extraction, architecture view standardization, and reference artifact validation. First, we use rule-based extraction to convert the standard-compliant course documents into the SRS format defined in this study, and we attach evidence trace links that map each extracted requirement back to its source location in the original document. Second, we use a model to convert the original architecture views into PlantUML representations, so that the reference architectures can be parsed and compared by the evaluation scripts. Two annotators then cross-validate each converted view against the original diagram to ensure structural consistency in component hierarchy, layer assignment, component relations, and overall structure. The corresponding extraction rules, prompts, and processing scripts are available in the project repository.

C. G-R2A: GitHub Repository Subset

G-R2A is designed to reflect architecture reasoning conditions in open-source projects. Compared with E-R2A, projects collected from GitHub usually involve more complex system

structures, richer technology stacks, longer evolution histories, and more diverse architectural representations. We use 373 manually coded architecture-view samples and their corresponding repositories from prior work [7] as the candidate pool, and retain repositories with sufficient supporting materials and parsable architecture views through a candidate scoring and manual inspection procedure, which yields 51 GitHub repository samples in the current version.

Candidate scoring and selection. We score each candidate repository with a deterministic, rule-based procedure that computes scores from repository metadata and content statistics along six dimensions: documentation scale, evidence-category coverage, requirement relevance, temporal consistency, traceability potential, and SRS convertibility. Candidates are retained based on preset thresholds on the total and per-dimension scores; the complete rubric, thresholds, and computation rules are released in our project repository for reproducibility. This stage initially selects 52 repositories; after manual inspection, one is removed, yielding 51 valid G-R2A samples.

Evidence collection. For each retained repository, we then assemble the source material from which its SRS will be reconstructed. We first resolve a source-code snapshot temporally aligned with its manually coded architecture view [7] when metadata is available, mitigating architecture drift caused by repository evolution. From this snapshot we collect the repository artifacts—which we refer to as *evidence*—spanning documentation (e.g., README, docs, requirements-like files) and implementation (e.g., interface definitions, entry points, configuration). Code artifacts are used to recover architecture-relevant interfaces, constraints, and deployment signals when documentation alone is insufficient.

Evidence pack construction. Unlike E-R2A, where requirements come from a single standardized document, a GitHub repository has no such specification: architecture-relevant information is sparse and scattered across many heterogeneous files. G-R2A therefore retrieves a small, traceable subset of evidence before generating the SRS.

The retrieval stage first cleans the collected files by removing noisy elements such as markup and links, and splits the cleaned text into evidence chunks. It then issues one query per SRS concern (e.g., functional requirements, non-functional requirements, constraints) over the target SRS schema, and combines three complementary signals over the same chunks: lexical matching via BM25 [19], vector-space matching via token-frequency cosine similarity with synonym expansion [20], and code-aware retrieval [21] that prioritizes architecturally relevant files (e.g., interface files for interface-related sections, configuration files for constraint-related ones). The three ranked lists are merged with reciprocal-rank fusion (RRF) [22]:

$$\text{RRF}_s(c) = \sum_{r \in \mathcal{T}} \frac{w_r}{k + \text{rank}_{r,s}(c)},$$

where $\mathcal{T}$ is the set of three retrieval signals, $\text{rank}_{r,s}(c)$ is the rank of chunk c under signal r for section s , w_r is the fusion weight of signal r , and k is a smoothing constant. We use heuristic weights ($w_{\text{BM25}}=1.0$, $w_{\text{vsm}}=0.9$, $w_{\text{code}}=0.8$; $k=60$)

TABLE I: Human review results for SRS quality control.

Decision Distribution				Agreement Summary	
Reviewer	A	PA	R	Metric	Value
R1	257	37	46	Full / majority agreement	288 / 52
R2	247	47	46	Avg. pair agreement	0.8980
R3	241	51	48	Fleiss' κ	0.7630

A=ACCEPT, PA=PARTIAL_ACCEPT, R=REJECT.

that prioritize exact lexical matching, since requirement and architecture terms often appear verbatim in documentation and identifiers; full settings are in the repository.

SRS generation and review. We use GPT-5.4 to generate an initial SRS from each evidence pack. The generation constraints enforce traceability and suppress hallucination: the SRS may use only facts from the evidence pack, every requirement must cite at least one evidence id, and unsupported requirements, assumptions, or design content are prohibited; claims inferred from code or configuration are kept distinguishable from those stated in documentation. A separate model, Claude Opus 4.8, then performs an automated review that flags candidate issues over a predefined set of issue types (e.g., unsupported claims, contradictions, missing requirements, traceability gaps).

To control SRS quality, the model-flagged issues undergo human review. Three reviewers, each with about four years of software development experience and familiarity with requirements and architecture documentation, first align on the criteria, then independently label each issue as ACCEPT, PARTIAL_ACCEPT, or REJECT. We report pairwise agreement and Fleiss' κ [23] (Table I); final decisions follow majority vote, with full disagreements resolved by discussion and recorded as adjudicated decisions. The resulting $\kappa = 0.76$ indicates substantial agreement. The SRS is then revised according to the final human decisions, producing the finalized specification used for architecture-view generation.

Reference view standardization. The steps above produce the SRS input for each repository; we now describe how its reference architecture view is prepared. Each retained repository has a manually coded architecture view from prior work [7]. Following the same procedure as E-R2A, a model converts each view into parsable PlantUML and two annotators cross-validate it against the original for consistency in component hierarchy, layer assignment, relations, and overall structure, yielding the reference views used for L1 evaluation. We treat each coded view as an authored reference at its recorded timestamp, to which the source-code snapshot and evidence are aligned, so comparisons reflect the architecture as documented rather than the latest repository state.

Benchmark statistics. Table II reports four structural properties of the reference views—*components* (architectural elements), *relations* (dependencies), their ratio (*relations per component*, i.e., coupling density), and *layers* (hierarchical depth)—characterizing how large, interconnected, and deeply structured each reference architecture is. E-R2A projects provide cleaner source requirements and more standardized reference views, but their reference views contain more

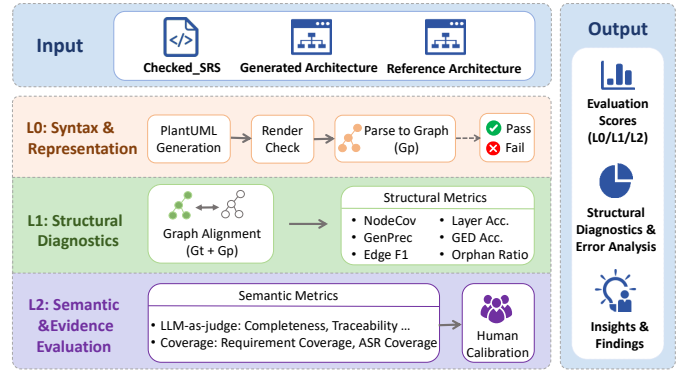


Fig. 2: Overview of the layered hybrid evaluation framework, comprising L0 syntax validation, L1 structural diagnostics, and L2 semantic and evidence evaluation.

TABLE II: Statistics of reference architecture views in R2ABENCH.

	Educational	GitHub	Overall
Projects	17	51	68
Components (avg.)	22.9	13.5	15.9
Relations (avg.)	8.8	13.5	12.4
Relations per component	0.38	1.00	0.78
Layers (avg.)	8.0	3.5	4.6

components and deeper layer structures (reflecting an explicitly modularized design). G-R2A projects, by contrast, often have fewer components but substantially denser relations (relations per component of 1.00 vs. 0.38). This relational density, rather than the number of components, is what characterizes the structural complexity of projects collected from GitHub.

IV. EVALUATION FRAMEWORK

A. Overview

We adopt a layered hybrid evaluation framework for the generated artifacts. Unlike code generation, architecture views cannot be validated through executable tests or `pass@k` metrics [24]: the same requirements may admit multiple reasonable representations, with no single executable oracle or uniquely correct diagram [3]. We evaluate the generated artifacts at three levels: L0 syntax and representation validity, L1 structural diagnostics, and L2 semantic and evidence-based quality—complemented by human calibration on a sampled subset.

At L0, we check whether the generated PlantUML artifact, which serializes the generated architecture view $\hat{V}_p$, can be rendered successfully and parsed into a directed architecture graph $G_p = (V_p, E_p)$. At L1, when a reference view V_p^* is available, we parse it into a reference graph $G_r = (V_r, E_r)$ and compare G_p against G_r using graph-based metrics such as node matching, edge matching, layer accuracy, and graph edit distance.

At L2, we use an LLM-as-a-Judge evaluator [17] to rate each view $\hat{V}_p$ along five quality dimensions (Table III).

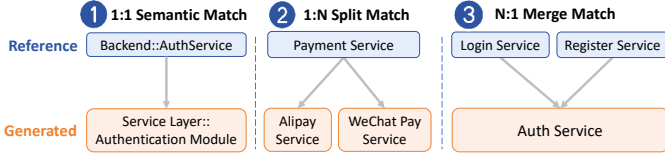


Fig. 3: Three forms of semantic correspondence under granularity mismatch: one-to-one, split, and merge between reference and generated components.

B. L0: Syntax and Representation Validity

L0 acts as a gate: it renders the PlantUML output and parses it into nodes and edges, and samples that fail either step are marked failed and excluded from all downstream graph-based metrics.

C. L1: Structural Diagnostics

L1 evaluates the structural similarity between the generated architecture view and the reference view. The evaluator parses both into directed architecture graphs $G_p = (V_p, E_p)$ and $G_r = (V_r, E_r)$, obtained from the generated view $\mathcal{V}_p$ and the reference view $\mathcal{V}_r^*$, respectively. Here, V represents architecture elements, such as services, components, databases, and external systems, while E represents directed dependencies or data-flow relations. We further write $V_{L,p} \subseteq V_p$ and $V_{L,r} \subseteq V_r$ for the leaf (component-level) nodes of each graph. Each parsed node retains its layer path, such as `Backend::AuthService`, enabling the evaluator to check not only whether a component is generated, but also whether it is placed in the correct architectural layer.

Because LLM-generated views may use component names or abstraction levels that differ from those in the reference view, L1 first uses GPT-5.4-mini to perform semantic alignment between generated nodes and reference nodes before computing graph metrics. Since internal nodes serve primarily as hierarchical grouping containers, alignment is restricted to leaf nodes, which represent the indivisible system components that carry architectural meaning. The alignment result is represented as:

$$M = \{(R_j, P_j, b_j)\}_{j=1}^m, \quad (3)$$

where $R_j \subseteq V_{L,r}$ denotes one or more reference leaf nodes, $P_j \subseteq V_{L,p}$ denotes one or more generated leaf nodes, and $b_j \in \{0, 1\}$ indicates whether the matched group is placed in a semantically correct layer.

Because the generated and reference views need not decompose the system at the same granularity, a reference component may map to a generated one as one-to-one, split (one reference realized by several generated components), or merge (several reference components collapsed into one), as illustrated in Fig. 3. The matched groups in M are then used to compute node coverage, generation precision, edge F1, layer accuracy, GED-based accuracy, and graph-structure anomaly metrics.

1) **Node Coverage and Generation Precision** ($NodeCov$, $GenPrec$): Because a generated view may realize the reference at a different abstraction level, we score nodes as a granularity-tolerant correspondence, read from the alignment M in two directions. A reference leaf $v \in V_{L,r}$

is covered ($c(v) = 1$) if it appears in some matched group of M , i.e. at least one generated node realizes its responsibility, and $c(v) = 0$ if it is unmatched (a missing component). Symmetrically, a generated leaf $u \in V_{L,p}$ is grounded ($g(u) = 1$) if it appears in some matched group, i.e. it is supported by a reference concept, and $g(u) = 0$ if it is unmatched (a hallucinated component). We report

$$NodeCov = \frac{1}{|V_{L,r}|} \sum_{v \in V_{L,r}} c(v), \quad (4)$$

$$GenPrec = \frac{1}{|V_{L,p}|} \sum_{u \in V_{L,p}} g(u). \quad (5)$$

$NodeCov$ and $GenPrec$ thus expose missing and hallucinated components respectively. Because a coarse generated node may cover several reference leaves and a finer set may jointly cover one, we report the two directional scores separately rather than as a single F1, keeping omission and over-generation distinguishable.

2) **Edge Correctness** ($F1_{edge}$): This metric evaluates whether the model correctly generates invocation or data-flow relations between components. A predicted edge is counted as correct if its source and target nodes correspond to the source and target nodes of a reference edge under the alignment M . Edge precision, recall, and F1 are defined as:

$$P_{edge} = \frac{TP_E}{TP_E + FP_E}, \quad R_{edge} = \frac{TP_E}{TP_E + FN_E}, \quad (6)$$

$$F1_{edge} = \frac{2P_{edge}R_{edge}}{P_{edge} + R_{edge}}. \quad (7)$$

Here, TP_E denotes matched edges, FP_E denotes unmatched generated edges, and FN_E denotes unmatched reference edges.

3) **Layer Accuracy** ($LayerAcc$): This metric evaluates whether matched components are placed in the correct architectural layer:

$$LayerAcc = \frac{1}{|M|} \sum_{j=1}^{|M|} b_j, \quad (8)$$

where $b_j = 1$ if the j -th matched group is placed in a correct architectural layer and $b_j = 0$ otherwise.

4) **Normalized GED Score** (GED_{acc}): This metric evaluates the overall topological similarity between G_p and G_r . To reduce the effect of graph size, we normalize the raw GED into an accuracy score from 0 to 100. Let $s_p = |V_p| + |E_p|$ and $s_r = |V_r| + |E_r|$:

$$GED_{acc} = 100 \cdot \max \left(0, 1 - \frac{GED(G_p, G_r)}{\max(s_p, s_r)} \right). \quad (9)$$

Here, $GED(G_p, G_r)$ denotes the graph edit distance between G_p and G_r ; a higher score indicates greater topological similarity.

TABLE III: L2 semantic and evidence-based evaluation dimensions.

Dimension	Evaluation Focus
Completeness	Whether the view covers key functional requirements, ASRs, external systems, data stores, constraints, and quality drivers.
Faithfulness	Whether generated components, relations, technologies, and assumptions are supported by source evidence or justified inference.
Architectural Rationality	Whether the decomposition, dependencies, responsibilities, boundaries, and quality-attribute handling are architecturally plausible.
Traceability	Whether architecture elements and decisions can be linked to requirements, ASRs, constraints, or evidence snippets.
Readability	Whether the view is understandable at the intended abstraction level, with clear naming, organization, boundaries, and relation labels.

5) **Orphan Ratio** (R_{orphan}): This metric measures the proportion of generated components with no incoming or outgoing edges:

$$R_{orphan} = \frac{|\{v \in V_{L,p} : \deg(v) = 0\}|}{|V_{L,p}|}. \quad (10)$$

D. L2: Semantic and Evidence Evaluation

L2 is the primary layer for evaluating architecture quality. It assesses whether the generated architecture is semantically reasonable, requirement-aware, and supported by available evidence. Specifically, L2 adopts an LLM-as-a-Judge evaluation strategy [17], implemented with GPT-5.4-mini, and rates each generated view $\hat{V}_p$ along five dimensions, organized with reference to established software architecture evaluation and documentation-quality criteria [3], [18], [5], [25]: completeness, faithfulness, architectural rationality, readability, and traceability. These dimensions are summarized in Table III.

Beyond these judge-rated dimensions, we additionally compute two judge-based coverage proxies that quantify how much of the requirement space is reflected in the generated view. Given a set of normalized requirements $\mathcal{R}$ and a generated view $\hat{V}_p$, requirement coverage is defined as:

$$ReqCov = \frac{|\{r \in \mathcal{R} : covered(r, \hat{V}_p) = 1\}|}{|\mathcal{R}|} \times 100\%, \quad (11)$$

where $covered(r, \hat{V}_p) = 1$ if the LLM judge determines that requirement r is addressed by at least one component or relation in the view. Similarly, for architecturally significant requirements $\mathcal{R}_{ASR}$:

$$ASRCov = \frac{|\{r \in \mathcal{R}_{ASR} : covered(r, \hat{V}_p) = 1\}|}{|\mathcal{R}_{ASR}|} \times 100\%. \quad (12)$$

Model-based judges are scalable but may be biased, so we calibrate the L2 evaluator against human judgments. The same three reviewers from the SRS review independently score a random sample of 272 views—drawn from the 866 that pass

TABLE IV: LLMs and agent frameworks for RQ1.

Category	Model
Open-source LLM	DeepSeek V3.2 Qwen3-Coder 480B-A35B
Closed-source LLM	Claude Sonnet 4.6 GPT-5
Agent Framework	MetaGPT [26] OpenHands [27] Mini-SWE [28]

L0 and spanning all model–framework configurations (95% confidence, 5% margin of error)—on the five L2 dimensions through a blind static review interface¹, following criteria agreed in advance. The L2 judge shows moderate alignment with the resulting human consensus, with 72.77% of scores within one point and a mean absolute error of 0.53 on the five-point scale.

V. EXPERIMENTS

A. Experimental Setup

Our empirical study is organized around three research questions:

- **RQ1 (Overall capability and input source).** How well do LLMs and agents generate architecture views, and how does this differ across input sources (educational vs. repository-derived)?
- **RQ2 (View characteristics).** How do the behavior and granularity of the target view affect generation quality on repository-derived projects?
- **RQ3 (Failure modes).** What recurring failure modes arise in requirements-to-architecture generation, in terms of structural errors, ASR coverage, NFR reasoning, traceability, and unsupported inference?

We evaluate a representative set of open-source models, closed-source models, and agentic frameworks (Table IV), pairing each framework with each model. MetaGPT is adapted as an agentic baseline for the R2A task—an SRSAnalyst→Architect→Reviewer→Refiner pipeline—rather than running its full development workflow. All agent outputs are evaluated with the same L0–L2 pipeline. All agents run under their default configurations. We use greedy decoding ($temperature = 0$) to reduce sampling variance and improve reproducibility; remaining configurations are released in our repository.

B. RQ1: Overall Capability and Input Source

We provide LLMs and agents with full SRSs and request PlantUML architecture views, evaluating their basic ability to derive diagrams from SRS, and we further examine how this ability differs across the two input sources.

Table V reports the overall results on R2ABENCH. Current LLMs and agentic frameworks generally produce syntactically valid views, but syntax validity remains strongly model- and

¹Review UI: <https://evalstudio-blind-review-272.netlify.app/>

TABLE V: Main results on R2ABENCH across the E-R2A and G-R2A subsets, with per-prediction rows pooled before aggregation. SV: syntax-valid rate. NodeCov / GenPrec: reference-side node coverage and generated-node precision. Layer: layer accuracy. GED Acc.: normalized graph-edit-distance accuracy. Orph.: fraction of generated leaf nodes with no incident edges (lower is better). Bold marks the best value per column; underline the best among the four frameworks for a given model.

Model	Framework	Layer 1: Structural Graph Metrics							Layer 2: Judge Scores				
		SV $\uparrow$	NodeCov $\uparrow$	GenPrec $\uparrow$	Edge F1 $\uparrow$	GED Acc. $\uparrow$	Layer $\uparrow$	Orph. $\downarrow$	Comp. $\uparrow$	Faith. $\uparrow$	Rat. $\uparrow$	Trace. $\uparrow$	Read. $\uparrow$
GPT-5	Direct	0.8971	0.7945	0.5347	0.0791	38.19	0.7496	5.88%	3.87	2.72	3.39	2.57	3.89
	MetaGPT	0.9559	0.8332	0.5112	0.1047	46.38	0.8047	<u>2.58%</u>	3.25	2.17	2.97	2.05	3.77
	Mini-SWE	0.8382	0.7696	<u>0.6193</u>	0.1320	<u>49.31</u>	0.7785	7.13%	3.42	2.51	3.16	2.16	3.96
	OpenHands	0.9265	0.7721	0.6147	0.0909	45.49	0.7677	10.63%	3.67	2.60	3.32	2.35	3.95
Claude Sonnet 4.6	Direct	0.9706	0.8490	0.5196	0.0804	43.52	0.8707	10.34%	<u>3.68</u>	<u>2.47</u>	<u>3.20</u>	<u>2.38</u>	3.76
	MetaGPT	0.8971	0.8201	0.5771	0.1008	48.26	0.7887	<u>3.20%</u>	3.39	2.23	3.13	2.15	3.79
	Mini-SWE	0.9118	0.7850	0.5225	0.1243	48.84	0.7472	3.96%	3.24	2.21	2.94	2.05	3.87
	OpenHands	1.0000	0.7541	<u>0.8949</u>	<u>0.1582</u>	<u>51.89</u>	<u>0.8964</u>	13.70%	3.60	2.41	3.14	2.23	<u>3.91</u>
DeepSeek V3.2	Direct	0.7059	0.7882	0.4958	0.1091	44.59	0.7628	13.00%	3.29	2.19	3.00	2.06	3.90
	MetaGPT	<u>0.7647</u>	0.7910	0.5969	0.1405	<u>50.27</u>	0.7670	<u>6.75%</u>	2.77	2.02	2.63	1.79	3.46
	Mini-SWE	<u>0.7647</u>	<u>0.8437</u>	0.5092	0.0704	45.93	0.8297	55.70%	3.08	2.08	2.85	1.94	3.60
	OpenHands	0.4853	0.8261	<u>0.9063</u>	0.0987	43.61	0.9259	35.96%	3.16	2.04	2.92	2.04	3.71
Qwen3-Coder 480B-A35B	Direct	0.5735	<u>0.7981</u>	0.6105	0.0877	47.85	0.7984	10.67%	<u>3.08</u>	<u>2.26</u>	<u>2.95</u>	<u>1.95</u>	<u>3.95</u>
	MetaGPT	0.7206	0.7284	0.5617	0.1057	49.12	0.7789	12.04%	2.67	1.98	2.39	1.63	3.41
	Mini-SWE	<u>0.7794</u>	0.6694	0.6413	0.1060	51.76	0.7549	6.89%	2.66	2.06	2.68	1.70	3.92
	OpenHands	0.5441	0.7926	0.9116	0.1758	52.82	<u>0.9228</u>	<u>4.54%</u>	2.97	2.14	2.84	1.86	<u>3.95</u>

TABLE VI: Source-wise performance shift from E-R2A subset to G-R2A subset. NodeCov is reference-side node coverage, and GenPrec is generated-node precision. The delta row reports G-R2A minus E-R2A.

Source	Rows	Valid	SV $\uparrow$	NodeCov $\uparrow$	GenPrec $\uparrow$	Edge F1 $\uparrow$	GED $\uparrow$	Layer $\uparrow$	L2 Avg. $\uparrow$	Req. Cov. $\uparrow$	ASR Cov. $\uparrow$	Unsup. Inf. $\downarrow$
E-R2A	272	211/272	0.7757	0.7682	0.6287	0.1562	50.53	0.7528	2.82	67.24%	49.39%	32.67%
G-R2A	816	655/816	0.8027	0.7947	0.6079	0.0947	46.26	0.8208	2.91	69.78%	58.13%	33.01%
G-E	+544	+444	+0.0270	+0.0265	-0.0209	-0.0615	-4.27	+0.0679	+0.09	+2.54%	+8.74%	+0.34%

framework-dependent. Averaged across the four generation settings, closed-source models retain a clear advantage over open-source ones (mean SV around 0.90–0.94 vs. 0.65–0.68). Framework choice compounds this: OpenHands reaches perfect SV with Claude Sonnet 4.6 yet is far less stable for the open-source models, dropping below 0.55.

Component recovery is consistently stronger than relation recovery. NodeCov ranges from 0.6694 to 0.8490, so valid outputs usually recover a substantial fraction of the reference components, whereas GenPrec varies far more widely (0.4958–0.9116). OpenHands markedly raises generated-node precision for Claude Sonnet 4.6, DeepSeek V3.2, and Qwen3-Coder, indicating fewer ungrounded components in its valid outputs. Because L1 metrics are conditional on L0-passing outputs, this precision must be read alongside SV: high precision among valid outputs does not by itself imply robust end-to-end generation.

Relation reconstruction is the hardest structural dimension: Edge F1 never exceeds 0.1758 and GED accuracy peaks at 52.82. The strongest structural configuration is Qwen3-Coder with OpenHands, which tops GenPrec, Edge F1, and GED accuracy; however, its low SV and weaker L2 scores qualify this structural lead.

To test whether low Edge F1 reflects a modeling limitation or a property of the task, we manually labeled a stratified random sample of 233 of the 589 missing edges. We find that 65 (27.9%, Wilson 95% CI [22.5%, 34.0%]) are not derivable from the SRS, while the remaining 168 (72.1%) are at least partially entailed by it; thus part of the gap is an information gap between requirements and the reference architecture. This does not explain the result away: restricting the reference

to directly derivable relations raises aggregate Edge F1 only to $\approx 11.0\%$, still far below node-level alignment. Relation recovery is therefore bounded by both incomplete architectural evidence in the input and limited relational modeling capability.

The semantic (L2) picture diverges from the structural one. GPT-5 with Direct prompting is the strongest semantic configuration, leading completeness, faithfulness, architectural rationality, and traceability; only readability goes instead to GPT-5 with Mini-SWE. The configurations that optimize structural similarity are not those that maximize semantic adequacy: Qwen3-Coder with OpenHands leads Edge F1 and GED accuracy yet trails GPT-5 Direct on every L2 dimension. Topological similarity is thus a useful but insufficient proxy for architecture quality.

Agentic frameworks help selectively rather than uniformly: each improves specific structural or stability metrics for some models (e.g., MetaGPT on GPT-5’s coverage and stability), but the gains neither transfer consistently across models nor extend to semantic quality, reshaping the error profile rather than delivering a reliable end-to-end advantage.

We further compare performance across the two input sources by pooling all configurations within each source, as shown in Table VI. Moving from E-R2A to G-R2A does not uniformly reduce quality. Syntax validity and the average L2 score both rise slightly (0.7757 $\rightarrow$ 0.8027 and 2.82 $\rightarrow$ 2.91), as does requirement coverage (+2.54 percentage points), while ASR coverage rises more clearly (+8.74 percentage points). With fewer components and shallower hierarchies (Table II), G-R2A places less burden on decomposition while still providing enough requirement information for models to produce semantically plausible and requirement-aware

TABLE VII: L0/L1 evaluation results by manually coded architecture-view categories. Cells report pass rate for L0 and mean values for L1 metrics. NodeCov reports reference-side node coverage, and GenPrec reports generated-node precision.

View factor	Category	L0 Pass	NodeCov	GenPrec	Edge F1	Layer	GED	Orphan
Granularity	High-level	79.1%	0.83	0.69	0.08	0.85	44.99	0.14
Granularity	Medium	81.2%	0.74	0.55	0.11	0.78	47.79	0.10
Granularity	Low-level	81.2%	0.88	0.52	0.06	0.91	44.94	0.06
Behavior	Static	82.8%	0.78	0.62	0.09	0.82	47.88	0.11
Behavior	Dynamic	75.0%	0.79	0.58	0.10	0.79	43.86	0.11
Behavior	Mixed	83.3%	0.90	0.63	0.10	0.91	44.96	0.11

views. The degradation is instead concentrated in relation-level structure: Edge F1 drops sharply ($0.1562 \rightarrow 0.0947$) and GED accuracy decreases ($50.53 \rightarrow 46.26$), whereas node-level metrics largely hold up—NodeCov even rises slightly ($0.7682 \rightarrow 0.7947$), GenPrec dips only marginally (-0.0209), and layer accuracy rises ($0.7528 \rightarrow 0.8208$), the last consistent with G-R2A’s shallower layer hierarchies. Models thus place components into plausible architectural regions even when they fail to recover the exact reference relations, mirroring the higher relational density of G-R2A reported in Table II.

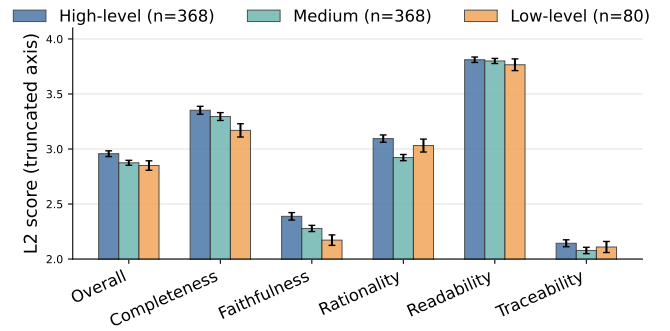
Finding 1: Current LLMs and agents identify components and layers far more reliably than they recover relations, and this gap is the dominant limitation across both settings. Moving to projects collected from GitHub degrades relation- and graph-level structure specifically, leaving semantic and evidence-oriented quality largely intact.

C. RQ2: View Behavior and Granularity

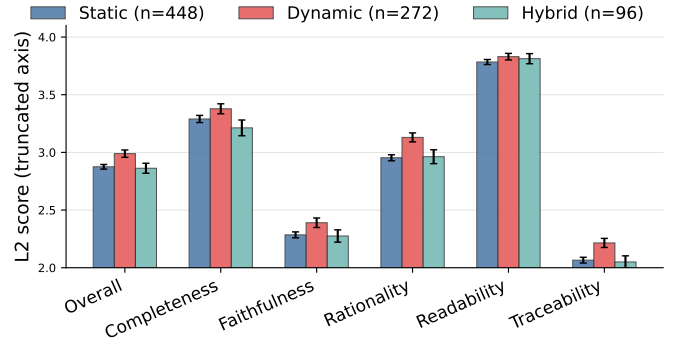
We use the manually coded metadata of the G-R2A architecture views to group samples by behavior and granularity, examining whether generation quality varies with the type and abstraction level of the target view. Behavior distinguishes static, dynamic, or mixed views, and granularity indicates a high, medium, or low level of detail.

Granularity. As shown in Fig. 4a, high-level views obtain the best semantic scores, leading on overall L2 and on all five L2 dimensions, while medium- and low-level views trail slightly. This suggests repository-derived generation is more reliable at higher abstraction, where the generator need not commit to concrete components and relations that are hard to infer faithfully from distributed artifacts. The L0/L1 metrics show a more mixed pattern: low-level views achieve the best NodeCov, layer accuracy, and orphan ratio, yet their GenPrec stays well below high-level views, so finer-grained targets still encourage unsupported components. Edge F1 remains low across all granularities (0.06 – 0.11).

Behavior. Behavior shows a similar split. Dynamic views obtain the strongest semantic quality, leading on overall L2 and all five dimensions over static and mixed views, plausibly because behavior-oriented views provide clearer cues about interactions and responsibilities. Structurally, however, mixed views perform best on several L0/L1 indicators and static views achieve the best GED accuracy, so dynamic views lead semantically without dominating the structural metrics.



(a) By architectural granularity



(b) By architectural behavior

Fig. 4: L2 semantic quality scores by architectural granularity (a) and behavior (b).

Across both factors, semantic and structural quality therefore emphasize different aspects of the target view. High-level and dynamic views receive stronger semantic judgments, whereas L0/L1 quality is metric-specific: low-level and mixed views improve component coverage and layer placement, while medium/static categories lead graph-edit similarity. Edge F1 stays low throughout (0.06 – 0.11 across granularity, 0.09 – 0.10 across behavior), so the component–relation gap from RQ1 persists across view categories.

Finding 2: Generation quality is higher for high-level and dynamic views in semantic evaluation, while structural quality varies by metric; across all categories, relation-level accuracy remains the main bottleneck.

D. RQ3: Failure Modes

We analyze recurring R2A failures across the three evaluation layers: rendering and parsing failures at L0, structural mismatches against the reference graph at L1, and semantic and evidence-related failures at L2. We identify them by combining metric outputs, judge explanations, and manual inspection of representative cases.

Applying the L0 validity gate, 866 of 1088 candidates passed and 222 failed, almost all due to ordinary PlantUML syntax or rendering errors rather than environment or dependency issues.

At L1, we decompose structural mismatches into five categories—node and edge omission, node and edge hallu-

cination, and layer error—over the 866 L1-evaluable outputs. Relation-level over-generation dominates: edge hallucination is the largest category (23,820 errors, 39.1%), followed by node hallucination (18,443, 30.2%). Edge and node omission account for 8,201 (13.5%) and 7,859 (12.9%), while layer errors are far rarer (2,666, 4.4%). Models thus more often introduce unsupported components and dependencies than misplace matched components across layers. As shown in Fig. 5(a), Claude Sonnet 4.6, GPT-5, and DeepSeek V3.2 share a similar over-generation pattern, whereas Qwen3-Coder has relatively higher omission rates.

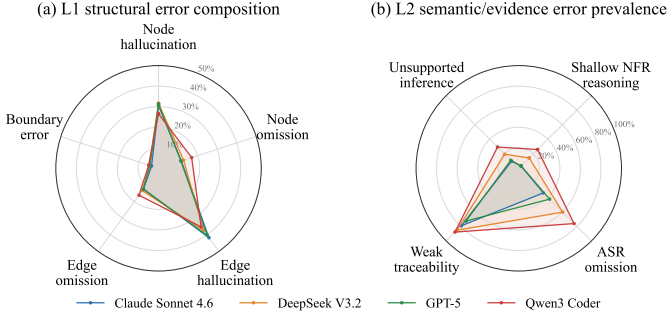


Fig. 5: Error analysis across models. (a) Distribution of the five L1 structural error categories; (b) prevalence of the four L2 semantic and evidence-related error categories.

TABLE VIII: L2 semantic and evidence-related error categories.

Error Type	Typical Symptom
ASR omission	Architecturally significant requirements are missing or not reflected in the view.
NFR neglect	Quality attributes are stated but not realized as concrete architecture mechanisms.
Weak traceability	Architecture elements cannot be linked to any requirement or evidence.
Unsupported claim	Plausible technologies or dependencies are introduced without supporting evidence.

At L2, we analyze semantic and evidence-related failures. Table VIII defines four recurring error categories that connect low L2 scores to concrete R2A failure modes. Unlike L1 graph errors, these categories are not mutually exclusive: a single view may, for instance, both omit an ASR and leave its components untraceable. We therefore report, in Fig. 5(b), the prevalence rate of each category—the proportion of L2-evaluable outputs in which it appears—rather than mutually exclusive counts.

Across models, the patterns are broadly similar: weak traceability and ASR omission are the most frequent failures, while unsupported inference and NFR neglect are comparatively rare—models fail less by inventing ungrounded elements than by failing to trace and fully cover requirements. Closed-source models perform better overall: Claude Sonnet 4.6 and GPT-5 show lower prevalence across most categories than Qwen3-Coder and DeepSeek V3.2, particularly on ASR omission and traceability. This likely reflects stronger long-context understanding, requirement abstraction, and evidence alignment in closed-source models.

To make these categories concrete, Fig. 6 presents an illustrative L2 error case whose reference intent is grounded in

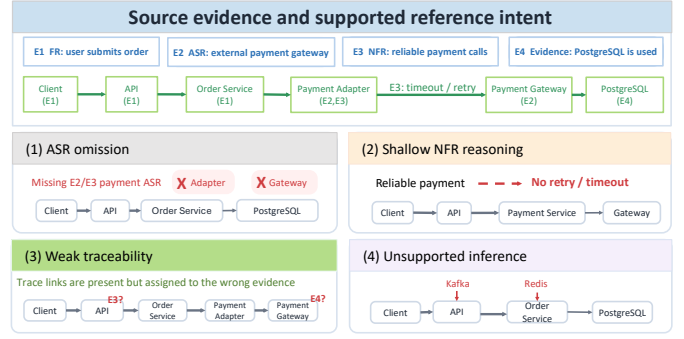


Fig. 6: L2 case study

four evidence snippets (order submission, an external payment gateway, a reliability requirement on payment calls, and PostgreSQL as the persistence technology). A faithful view should preserve the order-processing path, include the payment adapter and gateway, realize the reliability NFR through concrete mechanisms such as timeout and retry, and keep each element traceable. The lower panels show how a structurally plausible view still fails: omitting the payment ASR, reducing the reliability NFR to a vague label, mislinking evidence, or adding unsupported infrastructure (e.g., Kafka, Redis, a service mesh). This illustrates why L2 targets requirement coverage, NFR realization, traceability, and evidence grounding rather than graph-level similarity alone.

VI. DISCUSSION

A central lesson from our results is that graph-level agreement and architecture quality come apart. A view may match the reference structurally yet lack evidence support or miss key quality drivers, while two structurally different views can both be reasonable under ambiguous requirements or alternative decompositions. This is why R2ABENCH does not collapse quality into a single score: L1 serves as a reference-based diagnostic that localizes missing or hallucinated components, incorrect layer placement, and erroneous dependencies, while semantic adequacy, traceability, and ASR coverage—which our study finds are where models most often fail—are assessed separately at L2. Progress on R2A therefore cannot be read off diagram validity or topological similarity alone.

VII. RELATED WORK

A. Software Architecture Design and Architecture Views

Software architecture design connects requirements, quality attributes, and implementation constraints through explicit structural decisions. Classical architecture-design research emphasizes that architecture is not only a collection of components, but also a set of views, viewpoints, decisions, and tradeoffs that address stakeholder concerns [29], [1], [3]. Architecture-view notations and practices, including C4-style descriptions and quality-attribute-driven design, help architects choose an abstraction level, separate concerns, and reason about system qualities such as modifiability, performance, security, and reliability [30], [25], [31]. Recent empirical work on open-source architecture views further shows that architecture

information in repositories is heterogeneous and often spread across different representation styles [7]. Our work builds on this view-oriented tradition, but studies a different question: whether LLMs and agents can derive a traceable architecture view from requirements and repository evidence, and how the quality of such views can be measured.

B. LLMs for Software Architecture

LLMs are increasingly being explored for software architecture tasks, including design-decision support, pattern recognition, architecture documentation, and architecture generation from requirements [32], [8]. More focused studies have examined whether LLMs can generate architectural design decisions or ADR-like artifacts [33], transform informal specifications into UML component diagrams [13], map natural-language requirements to architecture choices with explicit trace links [14], and coordinate multiple LLM agents for C4 architecture modeling [34]. These studies indicate that LLMs can assist architecture work but expose recurring limitations: outputs still require expert review, evaluation settings are small or artifact-specific, and generated decisions often lack explicit evidence support. These limitations motivate treating requirements-to-architecture generation as a benchmarked, evidence-grounded task rather than a single prompt.

C. Benchmarks and Evaluation for Architecture Generation

A growing body of LLM-oriented benchmarks targets text-to-model generation, producing a specific artifact type from natural-language input, such as domain models [11], [35], UML class diagrams [9], sequence diagrams [10], [36], activity diagrams [37], and constrained graph models [38]. While these benchmarks establish useful task formulations and reference oracles, they usually rely on curated, self-contained descriptions that differ from realistic requirement documents, and they target individual diagram types rather than system-level architecture views. Closer to our setting, architecture-specific resources have begun to appear: Migliorini et al. mine architecture views from open-source repositories [7], recent studies evaluate LLM-based architecture-view generation directly from source-code repositories [12], and ArchBench proposes a platform for benchmarking generative AI on software-architecture tasks [39]. However, these resources mainly emphasize existing diagrams, source-code-to-view recovery, or benchmark infrastructure, offering limited support for requirement-grounded inputs and explicit requirement-to-architecture traceability.

Table IX makes this gap concrete, comparing representative datasets and benchmarks along source artifacts, target outputs, reference oracles, traceability support, and layered evaluation. No prior benchmark jointly couples realistic requirements, heterogeneous repository evidence, reference architecture views, and traceable multi-level evaluation—the gap R2ABENCH is designed to fill.

Evaluating generated architecture views is a related challenge, because architecture generation lacks a single executable oracle. Unlike repository-level coding benchmarks such as SWE-bench, where executable patch validation provides the main success

TABLE IX: Capability comparison of datasets and benchmarks for model and architecture generation.

Dataset / Benchmark	Primary source	Req.	Repo	View	Ref.	Trace	Eval
UML in GitHub [40]	GitHub UML artifacts	×	△	△	×	×	×
ModelSet [41]	GitHub / GenMyModel	×	×	×	△	×	×
Ext. EA ModelSet [42]	Curated ArchiMate	×	×	△	△	×	×
Golden UML ModelSet [43]	Curated UML cases	△	×	×	✓	×	△
Auto. Domain Modeling [11]	Domain descriptions	✓	×	×	✓	×	△
Ferrari et al. [35]	Requirements docs	✓	×	×	△	×	△
PLANTUCD [9]	Requirement–diagram	✓	×	×	✓	×	△
MermaidSeqBench [10]	Interaction prompts	✓	×	×	✓	×	△
MCeT [36]	Req. + sequence	✓	×	×	△	△	✓
LADEX [37]	Process descriptions	✓	×	×	✓	△	✓
Graph-model gen. [38]	Text-to-graph data	✓	×	×	✓	×	✓
Arch. views in OSS [7]	GitHub arch. views	×	△	✓	△	×	×
LLM view gen. [12]	Source-code repos	×	✓	✓	✓	×	△
ArchBench [39]	Aggregated arch. tasks	△	△	✓	△	△	△
R2ABENCH	Edu. + GitHub	✓	✓	✓	✓	✓	✓

✓ = primary capability; △ = partial / task-specific; × = not a primary capability. Req = requirement text; Repo = repository evidence; View = architecture view; Ref = reference oracle; Trace = trace/evidence links; Eval = layered evaluation.

signal [15], an architecture view may be syntactically valid yet incomplete, unsupported, or semantically weak, and the same requirements may admit multiple reasonable decompositions. Consequently, text-based similarity metrics are inadequate, while purely rule-based or single-judge LLM checks are either too rigid or vulnerable to bias and hallucination [13], [17]. These observations motivate a benchmark whose evaluation exposes multiple complementary dimensions—validity, structural consistency, semantic adequacy, and evidence support—rather than collapsing architecture quality into a single score.

VIII. THREATS TO VALIDITY

Internal validity. Two factors may bias the measured quality. First, our SRS generation, L1 alignment, and L2 judging all rely on GPT-5.4 series models, while GPT-5 is evaluated, raising a potential same-family preference; the L2 judge is calibrated against an independent human review of 272 sampled outputs, which provides a check on any systematic family bias. Second, G-R2A projects come from public GitHub and may appear in pretraining; yet if memorization dominated, G-R2A should score *higher* than E-R2A, whereas its relation-level metrics are lower (Edge F1 0.1562 $\rightarrow$ 0.0947; GED 50.53 $\rightarrow$ 46.26).

Construct validity. We operationalize architecture quality through syntax validity, structural diagnostics, and semantic and evidence-based assessment; these signals are complementary but read as diagnostic indicators rather than a complete measure of design rationale, trade-offs, or all acceptable decompositions.

External validity. R2ABENCH covers 68 projects from educational and GitHub sources, and the SRS—organized from repository documents and evidence—tends to be more complete than early-stage requirements in real development; our findings thus characterize R2A behavior within this scope, and broadening the domains and scale is left to future work.

IX. CONCLUSION

This paper presents R2ABENCH, a benchmark and layered evaluation framework for requirements-to-architecture reasoning that pairs educational projects with GitHub projects to span idealized and repository-grounded conditions. Our empirical study shows that current LLM-based systems can often generate syntactically valid and readable architecture

views, but relation-level architecture modeling remains a major bottleneck: models identify components more reliably than they recover dependencies. These findings suggest that R2A generation should be evaluated beyond diagram validity or graph similarity, with attention to semantic adequacy, evidence support and view granularity.

REFERENCES

- [1] B. Nuseibeh, “Weaving together requirements and architectures,” *Computer*, vol. 34, no. 3, pp. 115–119, 2001.
- [2] N. Medvidovic and R. N. Taylor, “Software architecture: foundations, theory, and practice,” in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering—Volume 2*, 2010, pp. 471–472.
- [3] “Ieee/iso/iec international standard for software, systems and enterprise—architecture description,” *ISO/IEC/IEEE 42010:2022(E)*, pp. 1–74, 2022.
- [4] N. Rozanski and E. Woods, *Software systems architecture: working with stakeholders using viewpoints and perspectives*. Addison-Wesley, 2012.
- [5] P. Clements, D. Garlan, R. Little, R. Nord, and J. Stafford, “Documenting software architectures: views and beyond,” in *25th International Conference on Software Engineering, 2003. Proceedings*. IEEE, 2003, pp. 740–741.
- [6] B. Tekinerdogan and H. Sözer, “Defining architectural viewpoints for quality concerns,” in *European Conference on Software Architecture*. Springer, 2011, pp. 26–34.
- [7] S. Migliorini, R. Verdecchia, I. Malavolta, P. Lago, and E. Vicario, “Architectural views: the state of practice in open-source software projects,” in *European Conference on Software Architecture*. Springer, 2024, pp. 396–415.
- [8] M. Esposito, X. Li, S. Moreschini, N. Ahmad, T. Cerny, K. Vaidhyathan, V. Lenarduzzi, and D. Taibi, “Generative ai for software architecture. applications, challenges, and future directions,” *Journal of Systems and Software*, p. 112607, 2025.
- [9] F. Alaswad, P. E. and B. Aljaddouh, “Plantucd: A dataset of software requirements and corresponding plantuml-based class diagrams,” 2026.
- [10] B. Shbita, F. Ahmed, and C. DeLuca, “Mermaidseqbench: An evaluation benchmark for llm-to-mermaid sequence diagram generation,” *arXiv preprint arXiv:2511.14967*, 2025.
- [11] K. Chen, Y. Yang, B. Chen, J. A. H. López, G. Mussbacher, and D. Varró, “Automated domain modeling with large language models: A comparative study,” in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2023, pp. 162–172.
- [12] M. Sathvika, R. Dhar, and K. Vaidhyathan, “Llm-based automated architecture view generation: Where are we now?” *arXiv preprint arXiv:2603.21178*, 2026.
- [13] A. Tagliaferro, S. Corbo, and B. Guindani, “Leveraging llms to automate software architecture design from informal specifications,” in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2025, pp. 291–299.
- [14] T. Helmi, “Arlo: A tailorable approach for transforming natural language software requirements into architecture using llms,” *arXiv preprint arXiv:2504.06143*, 2025.
- [15] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?, 2024,” URL <https://arxiv.org/abs/2310.06770>, vol. 7, 2023.
- [16] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.
- [17] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in neural information processing systems*, vol. 36, pp. 46 595–46 623, 2023.
- [18] ISO/IEC/IEEE, “Iso/iec/ieee international standard-systems and software engineering—life cycle processes—requirements engineering. iso/iec/ieee 29148: 2018 (e),” 2018.
- [19] S. Robertson and H. Zaragoza, *The probabilistic relevance framework: BM25 and beyond*. Now Publishers Inc, 2009, vol. 4.
- [20] G. Salton, A. Wong, and C.-S. Yang, “A vector space model for automatic indexing,” *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975.
- [21] S. Bajracharya, T. Ngo, E. Linstead, Y. Dou, P. Rigor, P. Baldi, and C. Lopes, “Sourcerer: a search engine for open source code supporting structure-based search,” in *Companion to the 21st ACM SIGPLAN symposium on Object-oriented programming systems, languages, and applications*, 2006, pp. 681–682.
- [22] G. V. Cormack, C. L. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, 2009, pp. 758–759.
- [23] J. L. Fleiss, “Measuring nominal scale agreement among many raters,” *Psychological bulletin*, vol. 76, no. 5, p. 378, 1971.
- [24] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [25] ISO/IEC, “ISO/IEC 25010:2023, Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuRE) – Product quality model,” <https://www.iso.org/standard/78176.html>, 2023, accessed: 2026-06-21.
- [26] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [27] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, “OpenHands: An Open Platform for AI Software Developers as Generalist Agents,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.16741>
- [28] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>
- [29] C. Hofmeister, P. Kruchten, R. L. Nord, H. Obbink, A. Ran, and P. America, “A general model of software architecture design derived from five industrial approaches,” *Journal of Systems and Software*, vol. 80, no. 1, pp. 106–126, 2007.
- [30] S. Brown, *The C4 Model: Visualizing Software Architecture*. ” O’Reilly Media, Inc.”, 2026.
- [31] R. Wojcik, F. Bachmann, L. Bass, P. Clements, P. Merson, R. Nord, and B. Wood, “Attribute-driven design (add), version 2.0,” Software Engineering Institute, Carnegie Mellon University, Tech. Rep., 2006.
- [32] L. Schmid, T. Hey, M. Armbruster, S. Corallo, D. Fuchß, J. Keim, H. Liu, and A. Koziolk, “Software architecture meets llms: A systematic literature review,” *arXiv preprint arXiv:2505.16697*, 2025.
- [33] R. Dhar, K. Vaidhyathan, and V. Varma, “Can llms generate architectural design decisions?-an exploratory empirical study,” in *2024 IEEE 21st International Conference on Software Architecture (ICSA)*. IEEE, 2024, pp. 79–89.
- [34] K. Szczepanik, J. Chudziak *et al.*, “Collaborative llm agents for c4 software architecture design automation,” *arXiv preprint arXiv:2510.22787*, 2025.
- [35] A. Ferrari, S. Abualhaija, and C. Arora, “Model generation with llms: From requirements to uml sequence diagrams,” in *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*. IEEE, 2024, pp. 291–300.
- [36] K. Ahmed, J. Song, B. Chen, O. Wei, and B. Zheng, “Mctet: Behavioral model correctness evaluation using large language models,” in *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2025, pp. 84–95.
- [37] P. Khamsepour, M. Cole, I. Ashraf, D. Tan, S. Puri, M. Sabetzadeh, and S. Nejati, “The impact of critique on llm-based model generation from natural language: The case of activity diagrams,” *arXiv preprint arXiv:2509.03463*, 2025.
- [38] B. Chen, O. Wei, B. Zheng, and G. Mussbacher, “Accurate and consistent graph model generation from text with large language models,” in *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2025, pp. 130–141.
- [39] B. Adnan, A. Gupta, S. Akshathala, and K. Vaidhyathan, “Archbench: Benchmarking generative-ai for software architecture tasks,” *arXiv preprint arXiv:2603.17833*, 2026.

- [40] G. Robles, T. Ho-Quang, R. Hebig, M. R. Chaudron, and M. A. Fernandez, "An extensive dataset of uml models in github," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 519–522.
- [41] J. A. H. López, J. L. Canovas Izquierdo, and J. S. Cuadrado, "Modelset: a dataset for machine learning in model-driven engineering," *Software and Systems Modeling*, vol. 21, no. 3, pp. 967–986, 2022.
- [42] P.-L. Glaser, E. Sallinger, and D. Bork, "The extended ea modelset—a fair dataset for researching and reasoning enterprise architecture modeling practices," *Software and Systems Modeling*, pp. 1–19, 2025.
- [43] C. Verbruggen, L. Netz, P.-L. Glaser, M. Scholz, C. Huemer, M. Calamo, B. Rumpe, M. Snoeck, and D. Bork, "Toward a community-curated golden dataset of uml models," in *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 2025, pp. 43–50.